

Check Completeness of a Binary Tree

Difficulty: Medium

Patterns: Tree, Binary Tree, Breadth-First Search

LeetCode: <https://leetcode.com/problems/check-completeness-of-a-binary-tree/>

1. Problem Summary

Given the `root` of a binary tree, determine whether it is a complete binary tree.

A complete binary tree has two rules:

1. Every level before the last level must be completely filled.
2. Nodes on the last level must appear as far left as possible.

Examples:

- `root = [1,2,3,4,5,6]` returns `true`
- `root = [1,2,3,4,5,null,7]` returns `false` because `7` appears after a missing left child slot.

2. Constraints and Observations

- Constraint: The number of nodes is in `[1, 100]`.
- What it implies: Any straightforward `O(n)` traversal is easily fast enough.
- Constraint: `1 <= Node.val <= 1000`.
- What it implies: Values do not matter. Completeness depends only on tree shape.
- Observation: Completeness is naturally checked in level order, from left to right.
- Observation: Once a missing child slot appears in level order, every later slot must also be missing. If a real node appears after that gap, the tree is not complete.

3. Pattern Recognition

This is a binary tree BFS problem.

The phrase "every level" points toward level-order traversal. The phrase "as far left as possible" tells us we must preserve left-to-right order within each level.

The best mental model is to imagine the tree serialized level by level, including missing child slots:

```
[1, 2, 3, 4, 5, null, 7]
```

In a complete tree, once this sequence reaches the first `null`, there should be no later non-null node.

4. Naive Approach

A first idea is to explicitly count nodes per level:

1. Traverse each level.
2. Check that every level before the last has the maximum possible number of nodes.
3. For the last level, check that all nodes are packed to the left.

This can work, but it is awkward. You need to know whether a level is the last level, track missing positions, and handle cases where a child is missing in the middle.

The bottleneck is not time complexity. The bottleneck is implementation complexity: we are trying to separately reason about levels when level-order slot order already contains the answer.

5. Key Intuition

Completeness fails exactly when there is a "hole" before a real node.

During BFS:

- Visit nodes from top to bottom.
- Within each level, visit from left to right.
- Push both children, even when a child is `None`.

Once BFS sees `None`, it means a slot is missing. From that point onward, every later item in BFS order must also be `None`. If we later see a real node, that real node is too far right, so the tree is not complete.

6. Optimal Solutions Ranked

Variant 1: BFS With Null Slots

- Idea: Put nodes and `None` placeholders into a queue. Once a `None` is popped, every later popped item must also be `None`.
- Time: $O(n)$.
- Space: $O(n)$.
- Tradeoff: Very direct and easy to explain. It may enqueue extra `None` placeholders, but still linear for this problem.

Variant 2: BFS With `seen_missing_child` Flag

- Idea: Process only real nodes. If any node is missing a child, set a flag. After that, no later node may have any child.
- Time: $O(n)$.
- Space: $O(n)$.
- Tradeoff: Avoids storing `None` placeholders, but has more branching logic.

Variant 3: Index-Based BFS

- Idea: Assign array-style indices: root is `1`, left child is `2*i`, right child is `2*i+1`. A complete tree with `n` nodes should have maximum index `n`.
- Time: $O(n)$.
- Space: $O(n)$.
- Tradeoff: Elegant and useful for heap intuition, but index growth is unnecessary here and can overflow in fixed-width languages for very deep trees.

Most optimal practical choice: BFS with null slots.

No solution can beat $O(n)$ worst-case time because a complete tree may require inspecting every node before returning `true`.

7. Most Optimal Approach

Use BFS and preserve missing child slots.

Maintain a queue initialized with `root`. Also maintain a boolean `seen_null`.

For each item popped:

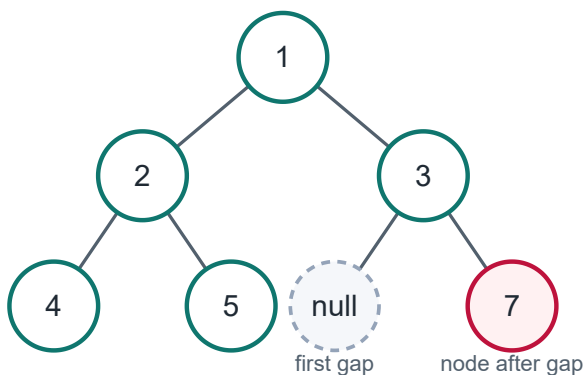
- If it is `None`, set `seen_null = True`.
- If it is a real node and `seen_null` is already true, return `False`.
- Otherwise enqueue its left child and right child.

If the traversal finishes without seeing a real node after a gap, return `True`.

8. Algorithm

1. Initialize `queue = deque([root])`.
2. Initialize `seen_null = False`.
3. While the queue is not empty:
 1. Pop the leftmost item.
 2. If the item is `None`, set `seen_null = True`.
 3. Otherwise, if `seen_null` is already `True`, return `False`.
 4. Otherwise, append `node.left` and `node.right` to the queue.
4. Return `True`.

9. Visual Explanation



Completeness rule:
After level-order traversal sees
`null`, every later slot must also be `null`.

Level-order slots:

1

2

3

4

5

`null`

7

Seeing node 7 after the first `null` means the last level is not filled from left to right.

Caption: In level order, a complete tree cannot have a real node after the first missing slot.

For `root = [1,2,3,4,5,null,7]`, the traversal sees:

```
1, 2, 3, 4, 5, null, 7
```

The `null` is the first missing slot. Seeing `7` afterward proves the last level is not filled from left to right.

10. Dry Run

Use the incomplete example:

```
root = [1,2,3,4,5,null,7]
```

Step	Popped	seen_null before	Action	Queue effect
1	1	False	enqueue 2 , 3	no gap yet
2	2	False	enqueue 4 , 5	no gap yet
3	3	False	enqueue None , 7	missing left child is now in queue
4	4	False	enqueue its missing children	still valid so far
5	5	False	enqueue its missing children	still valid so far
6	None	False	set <code>seen_null = True</code>	first gap found
7	7	True	real node after gap	return <code>False</code>

The algorithm rejects the tree exactly when it sees a real node after the first missing slot.

11. Correctness Argument

Invariant:

During BFS, before a real node is processed, `seen_null` is true if and only if an earlier missing slot has already appeared in level-order position.

Why this works:

- BFS processes slots in the same order that a complete-tree array representation would use: top to bottom, left to right.
- In a complete binary tree, all real nodes must appear before all missing slots in that order.
- If the algorithm sees `None`, it records that the missing-slot region has begun.
- If a later real node appears after that, the real node is positioned to the right of a gap, violating completeness.
- If no real node appears after the first missing slot, then all existing nodes occupy the earliest level-order positions, so every level before the last is full and the last level is left-packed.

Therefore, the algorithm returns `True` exactly for complete binary trees.

12. Complexity Analysis

- Time: $O(n)$, where n is the number of real nodes.
- Why: Each real node is popped once, and each child slot is considered once.
- Space: $O(n)$ for the queue in the worst case.

Even though this variant enqueues `None` slots, it enqueues at most two child slots per real node, so the total work remains linear.

13. Edge Cases

- Single node: no gap before a real node, return `True`.
- Last level partially filled from the left: return `True`.
- Missing left child but existing right child at the same parent: return `False`.
- Missing child in one subtree followed by later nodes elsewhere: return `False`.
- Perfect binary tree: no gap until after all real nodes, return `True`.

14. Approach Code: Flag Without Enqueuing Nulls

```
from collections import deque
from typing import Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isCompleteTree(self, root: Optional[TreeNode]) -> bool:
        queue = deque([root])
        seen_missing_child = False

        while queue:
            node = queue.popleft()

            if node.left:
                if seen_missing_child:
                    return False
                queue.append(node.left)
            else:
                seen_missing_child = True

            if node.right:
                if seen_missing_child:
                    return False
                queue.append(node.right)
            else:
                seen_missing_child = True

        return True
```

15. Approach Code: Index-Based BFS

```
from collections import deque
from typing import Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isCompleteTree(self, root: Optional[TreeNode]) -> bool:
        queue = deque([(root, 1)])
        count = 0
        last_index = 0

        while queue:
            node, index = queue.popleft()
            count += 1
            last_index = index

            if node.left:
                queue.append((node.left, 2 * index))
            if node.right:
                queue.append((node.right, 2 * index + 1))

        return count == last_index
```


16. Final Clean Code

```
from collections import deque
from typing import Optional

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

class Solution:
    def isCompleteTree(self, root: Optional[TreeNode]) -> bool:
        queue = deque([root])
        seen_null = False

        while queue:
            node = queue.popleft()

            if node is None:
                seen_null = True
                continue

            if seen_null:
                return False

            queue.append(node.left)
            queue.append(node.right)

        return True
```

17. Key Takeaways

- Main pattern: BFS level-order traversal.
- Main trick: Include missing child slots in the traversal order.
- Core invariant: after the first `None`, every later slot must also be `None`.
- Interview explanation: "A complete tree has no holes before real nodes in level order."